OmniSIG Engine API

The OmniSIG Engine features a remotely accessible API over ZMQ using the request-reply model. OmniSIG Engine listens for commands on a "REP" socket type. Requests are sent from an external process using a "REQ" socket type. See the ZMQ documentation for further details on these socket types.

The API "REP" socket defaults to listen on all network interfaces using TCP port 4003 by default. This interface can be changed using the `-z` or `--zmqapi` option to specify any valid ZMQ endpoint. See ZMQ Connect Documentation for details. The default behavior can be replicated with the arguments `-z tcp://0.0.0.0:4003`. In this case `tcp://` is the ZMQ endpoint type, `0.0.0.0` specifies that all interfaces can receive requests, and `4003` is the port number.

As another example, the API can be restricted to only receiving commands from the local machine using the same port with the argument `-z tcp://127.0.0.1:4003`

IPC endpoints can also be used. For example to use an IPC file named omnisig.api use

```
-z ipc:///license/omnisig.api
```

Included API Utility

Included in your OmniSIG Engine package is a convenience Python utility for handling some of the book-keeping associated with opening the correct socket type. This utility is called `omnisigapi.py`. You can either run it as a standalone utility or import it as a Python3 module.

To run it as a standalone program:

```
python omnisigapi.py
```

This will open up a ZMQ socket at `0.0.0.0:4003` to communicate with OmniSIG Engine running locally over port 4003. This is the default port for the OmniSIG Engine API. If you start OmniSIG Engine with a different port, or are attempting to communicate with OmniSIG Engine from a different machine, you can run the program as follows:

```
python omnisigapi.py -p <omnisig_api_port> -i
<address_of_machine_running_omnisig>
```

This should display a list of the available function calls, as well as a prompt:

```
deepsig>
```

Here you can enter commands (they will tab-complete). To send query commands with no parameters, such as getters:

```
deepsig> wideband_inference.get_confidence_threshold
0.500000
```

To send commands with a parameter, such as a setter:

```
deepsig> wideband_inference.set_confidence_threshold 0.75
```

```
 deepsig> inference.set_wb_freq_ranges [[268e6,269e6],[880e6,882e6]]
```

```
 deepsig> inference.get_wb_freq_ranges
```

Note that booleans should be sent with non-capitalized `true` or `false` when using the ulitity on its own.

To import the API utility as a module and use it elsewhere:

```
import omnisigapi as ds_api

# note that the following assumes the OmniSIG Engine ZMQ API is listening
locally at the default port 4003.
# if a different address was chosen when launching OmniSIG Engine, that will
need to be reflected in the following command
api = ds_api.omnisig_api(4003, "0.0.0.0")

api.send("help")
```

When sending a JSON argument while importing omnsigiapi as a module, it is recommended to use `json.dumps` for readability, i.e.:

```
json_dict ={
        "descriptions":["LTE,5G"],
        "enabled":True,
        "max_bandwidth":20.0e6,
        "max_rssi":100.0,
        "min_bandwidth":5.0e6,
        "min_confidence":0.0,
        "min_rssi":-100.0
}

api.send("recording.set_trigger_config", json.dumps(json_dict))
```

If not using `json.dumps`, booleans in JSON arguments must be uncapitalized (i.e. `true` or `false`), and the JSON should be escaped with single quotes:

```
api.send("recording.set_trigger_config", '{"descriptions":
["5G"],"enabled":true,"max_bandwidth":20.0e6,"max_rssi":100.0,"min_bandwidth":5.0
```

NOTE: The included API utility has only been tested with Python 3.6. Other versions may not be supported using the example utility.

Available API Functionality

The ZMQ API uses strings to encode requests and responses. Each command is either a getter or a setter. Getters will accepted a single string value of the value to get. The value is returned as a string. Setters will accept a string value of the value to set and a string representation of the value to set. Setters will return an empty string.

The following section describes the current commands and their usage

---

Help

Returns a human-readable list of the functions currently available to you and type hints for input and return parameters.

Usage:

```
api.send("help")
```

There is also a function to return a JSON array of function details.

Usage:

```
api.send("details")
```

The returned array will have the following form.

```
[
  {
    "arguments": [
      {
        "type": "void"
      }
    ],
    "name": "details",
    "returnType": "string"
  },
  ...
]
```

Version

Return a string indicating the currently running version of OmniSIG.

Usage:

```
api.send("version")
```

There are also convenience functions that return the major, minor, and patch version numbers individually. Usage:

```
api.send("version_major")
api.send("version_minor")
api.send("version_patch")
```

Sample Buffer

Get Channelizer Config (`buffer.get_channelizer_config`)

Returns the current channelizer JSON config.

See Channelizer Settings Tab for more details.

Usage:

```
api.send("buffer.get_channelizer_config")
```

Get Flush Samples (`buffer.get_flush_samples`)

Return the sample flush period in ms

Usage:

```
api.send("buffer.get_flush_samples")
```

Get Multichannel Config (`buffer.get_multichannel_config`)

Returns the multichannel JSON config.

Usage:

```
api.send("buffer.get_multichannel_config")
```

Get Number of RX Channels (`buffer.get_num_rx_chans`)

Returns the number of available RX channels for the current source.

Usage:

```
api.send("buffer.get_num_rx_chans")
```

Get Scanner Delay (`buffer.get_scanner_delay`)

Gets the time to pause at each interval (in ms).

Usage:

```
api.send("buffer.get_scanner_delay")
```

Get Scanner Dwells (`buffer.get_scanner_dwells`)

Get the current list of scanner hopping frequencies.

Usage:

```
api.send("buffer.get_scanner_dwells")
```

Get Scanner Start (`buffer.get_scanner_start`)

Gets the lower frequency bound for the radio scanner (in Hz) operating in a linear sweep.

Usage:

```
api.send("buffer.get_scanner_start")
```

Get Scanner State (`buffer.get_scanner_state`)

Returns `True` if OmniSIG is currently operating in scanning mode -- otherwise returns `False`

Usage:

```
api.send("buffer.get_scanner_state")
```

Get Scanner Stop (`buffer.get_scanner_stop`)

Gets the upper frequency bound for the radio scanner (in Hz) operating in a linear sweep.

Usage:

```
api.send("buffer.get_scanner_stop")
```

Get Size (`buffer.get_size`)

Gets the buffer size in number of samples.

Usage:

```
api.send("buffer.get_size")
```

Get Stride (`buffer.get_stride`)

Gets the buffer stride (sampling interval of inference frames from the buffer)

Usage:

```
api.send("buffer.get_stride")
```

Load Multichannel Config (`buffer.load_multichannel_config`)

Loads the multichannel config once it has been set explicitly (either by raw JSON or file path)

Usage:

```
api.send("buffer.load_multichannel_config")
```

Reset (`buffer.reset`)

Reset the underlying buffer

Usage:

```
api.send("buffer.reset")
```

Set Flush Samples (`buffer.set_flush_samples`)

Set the sample flush period in ms.

Usage:

```
api.send("buffer.set_flush_samples", 10)
```

Set Multichannel Config (`buffer.set_multichannel_config`)

Set the multichannel JSON config explicitly.

Usage:

```
json_dict = {
    "calibration_table": [
        {
            "frequency": 860000000.0,
```

```
            "phases":[0,0.2318],
            "amplitudes": [1, 0.984]
        },
        {
            "frequency": 900000000.0,
            "phases":[0, 1.0],
            "amplitudes":[1, 0.99]
        }
    ]
}
api.send("buffer.set_multichannel_config", json.dumps(json_dict))
```

Set Multichannel Config Path (`buffer.set_multichannel_config_path`)

Sets the multichannel config via a path to a JSON config file.

Usage:

```
api.send("buffer.set_multichannel_config_path",
"/path/to/multichannel_config.json")
```

Set Scanner Delay (`buffer.set_scanner_delay`)

Sets the time to pause at each interval (in ms).

Usage:

```
api.send("buffer.set_scanner_delay", "1000")
```

Set Scanner Dwells (`buffer.set_scanner_dwells`)

Specify a list of frequencies to hop between. The frequencies should be comma-separated and given in MHz. As an example, the following would hop between 800 MHz, 900 MHz, and 1000 MHz.

Usage:

```
api.send("buffer.set_scanner_dwells", "800,900,1000")
```

Set Scanner Start (`buffer.set_scanner_start`)

Sets the lower frequency bound for the radio scanner (in Hz) which will operate in a linear sweep. Must be less than the scanner stop frequency.

Usage:

```
api.send("buffer.set_scanner_start", "700e6")
```

Set Scanner Stop (`buffer.set_scanner_stop`)

Sets the upper frequency bound for the radio scanner (in Hz) which will operate in a linear sweep. Must be greater than the scanner start frequency.

Usage:

```
api.send("buffer.set_scanner_stop", "840e6")
```

Set Size (`buffer.set_size`)

Sets the buffer size in number of samples.

Usage:

```
api.send("buffer.set_size", 128*512*10)
```

Set Stride (`buffer.set_stride`)

Sets the buffer stride (sampling interval of inference frames from the buffer)

Usage:

```
api.send("buffer.set_stride", 5)
```

Start (`buffer.start`)

Starts/resumes the buffer source.

Usage:

```
api.send("buffer.start")
```

Start Scan (`buffer.start_scan`)

Starts/resumes radio scanner.

Usage:

```
api.send("buffer.start_scan")
```

Stop (`buffer.stop`)

Stops/pauses the buffer source.

Usage:

```
api.send("buffer.stop")
```

Stop Scan (`buffer.stop_scan`)

Stops/pauses radio scanner.

Usage:

```
api.send("buffer.stop_scan")
```

Inference

Get Channel (`inference.get_channel`)

Gets the current inference channel.

Usage:

```
api.send("inference.get_channel")
```

Set Channel (`inference.set_channel`)

Sets the current inference channel.

Usage:

```
api.send("inference.set_channel", 1)
```

Set Model (`inference.set_model`)

Set/reset the OmniSIG model during runtime using an absolute path to a model file on the host machine. Enables hot-swapping models without incurring the OmniSIG initialization overhead, as would be the case in a stop/start/restart type workflow.

Usage:

```
api.send("inference.set_model", "/path/to/model.ds")
```

Start (`inference.start`)

Starts inference.

Usage:

```
api.send("inference.start")
```

Stop (`inference.stop`)

Stops inference.

Usage:

```
api.send("inference.stop")
```

Logger

When integrating external applications with OmniSIG, users may find it useful to be able to inject log messages into the OmniSIG logger output (as opposed to / in addition to its own logger infrastructure). These messages can be formatted as one would typically format Python strings to include variables, formatting, etc.

Log Info (`logger.log_info`)

Usage:

```
api.send("logger.log_info", "This is an INFO message")
```

Log Warn (`logger.log_warn`)

Usage:

```
api.send("logger.log_warn", "This is a WARNING message")
```

Log Error (`logger.log_error`)

Usage:

```
api.send("logger.log_error", "This is an ERROR message")
```

Publishers

List Branches (`publisher[0].list_branches`)

Lists all branches of the branch publisher it is called on (specified by the ID in brackets). This function is only available to call on branch publishers, so check that it is available with the `help` function before using it. Use `publishers.list` to check which publisher corresponds with which ID.

Usage:

```
api.send("publisher[0].list_branches")
```

List Filters (`publisher[0].list_filters`)

Lists any filters set on the publisher specified by the ID in brackets (you may use `publishers.list` to see which publisher corresponds with which ID). It will only show filters applied to specific publishers, not filters applied to all publishers (such as the Confidence Filter set to 0.75 by default).

Usage:

```
api.send("publisher[0].list_filters")
```

Make Branch (`publisher[0].make_branch`)

Adds a new branch to the branch publisher (specified with the ID in brackets-- use `publishers.list` to check which publisher corresponds with which ID). This function is only available to call on branch publishers, so check that it is available with the `help` function before using it.

The branch spec should be a JSON of a publisher

Usage (new branch containing file publisher):

```
branch_json ={
    "type": "file",
    "path": "/valid_path/file1"
}

api.send("publisher[0].make_branch", json.dumps(branch_json))
```

Make Filter (`publisher[0].make_filter`)

Makes and applies a new filter, from a provided JSON spec, to the publisher specified by the ID in brackets (you may use `publishers.list` to see which publisher corresponds with which ID).

Usage (Confidence Filter):

```
filter_json ={
        "type": "confidence",
        "min_confidence": 0.9,
        "min_unknown_confidence": 0.5
}

api.send("publisher[0].make_filter", json.dumps(filter_json))
```

Usage (RSSI Filter):

```
filter_json ={
        "type": "rssi",
        "min_rssi": -10,
        "max_rssi": 10
```

```
}

api.send("publisher[0].make_filter", json.dumps(filter_json))
```

See Postprocess Filters for more details on each parameter for the RSSI and Confidence filters.

Usage (Labels Filter):

```
filter_json ={
        "type": "label",
        "label": ["LTE","5G"]
}

api.send("publisher[0].make_filter", json.dumps(filter_json))
```

When used, the Labels filter will filter out any signal labels not specified in the list of labels provided. I.e. this one would filter out any labels that are not LTE or 5G.

Usage (ITU Guardrail Filter):

```
filter_json ={
    "type": "itu_guardrail",
    "mark_unknown":True,
    "signal_list": [
        {
            "signal_id": "BlueTooth",
            "ranges": [
                {
                    "lower_freq_bound": 0,
                    "upper_freq_bound": 2420e6
                }
            ]
        },
        {
            "signal_id": "WIFI",
            "ranges": [
                {
                    "lower_freq_bound": 2420e6,
                    "upper_freq_bound": 24100e6
                }
            ]
        }
    ]
}

api.send("publisher[0].make_filter", json.dumps(filter_json))
```

See ITU Guardrail Filter to learn about the parameters and the default ITU guardrail filter. You may set the parameters to non-default values using the API, like in the usage above.

Remove Branch (`publisher[0].remove_branch`)

Removes a branch from a branch publisher. The publisher is specified with the ID in brackets, and the ID must correspond with a branch-type publisher. This function is only available to call on branch publishers, so check that it is available with the `help` function before using it.

The function takes one argument, the ID of the branch to be removed. You may use `publisher[0].list_branches` to check what branch corresponds with what ID.

Usage (to remove branch with ID 1 on publisher with ID 0):

```
api.send("publisher[0].remove_branch", "1")
```

Remove Filter (`publisher[0].remove_filter`)

Removes a filter from a publisher (specified by the number in brackets). You may use the API calls `publishers.list` and `publisher[0].list_filters` to check the ID of the publisher and filter you want to specify.

Usage (to remove filter with ID 4 on publisher with ID 0):

```
api.send("publisher[0].remove_filter", "4")
```

Disable All (`publishers.disable`)

Disables all publishers. When publishers are disabled, they will temporarily stop publishing data. Can be reverted by `publishers.enable`.

Usage:

```
api.send("publishers.disable")
```

Enable All (`publishers.enable`)

Enables all publishers. When publishers are enabled, they will publish data. Can be reverted by `publishers.disable`.

Usage:

```
api.send("publishers.enable")
```

Get Average Post Publish Latency in Microseconds (`publishers.get_avg_post_publish_latency_us`)

Gets average post-publish latency in microseconds.

Usage:

```
api.send("publishers.get_avg_post_publish_latency_us")
```

Get Average Pre Publish Latency in Microseconds (`publishers.get_avg_pre_publish_latency_us`)

Gets average pre-publish latency in microseconds.

Usage:

```
api.send("publishers.get_avg_pre_publish_latency_us")
```

Get Average Publish Duration in Microseconds (`publishers.get_avg_publish_duration_us`)

Gets average publish duration in microseconds.

Usage:

```
api.send("publishers.get_avg_publish_duration_us")
```

Get Max Post Publish Latency in Microseconds (`publishers.get_max_post_publish_latency_us`)

Gets maximum post-publish latency in microseconds.

Usage:

```
api.send("publishers.get_max_post_publish_latency_us")
```

Get Max Pre Publish Latency in Microseconds (`publishers.get_max_pre_publish_latency_us`)

Gets maximum pre-publish latency in microseconds.

Usage:

```
api.send("publishers.get_max_pre_publish_latency_us")
```

Get Max Publish Duration in Microseconds (`publishers.get_max_publish_duration_us`)

Gets maximum publish duration in microseconds.

Usage:

```
api.send("publishers.get_max_publish_duration_us")
```

List All (`publishers.list`)

Displays a list of the currently active publishers and their unique publisher ID number.

Usage:

```
api.send("publishers.list")
```

Make (`publishers.make`)

Creates a new publisher by URI. Results will begin publishing to the new publisher immediately.

Example to add a new ZMQ publisher:

```
api.send("publishers.make", "zmq://tcp://0.0.0.0:5000")
```

Remove (`publishers.remove`)

Removes publisher specified with provided ID. Use `publishers.list` to check which publisher corresponds with which ID.

Usage (remove publisher with ID 1):

```
api.send("publishers.remove", "1")
```

Reset Timing (`publishers.reset_timing`)

Resets timing for publisher related latency and duration measurements.

Usage (remove publisher with ID 1):

```
api.send("publishers.reset_timing")
```

Set Demod Annotations (`publishers.set_demod_annotations`)

Set a list of demod fields that will be correlated with active detections.

Usage:

```
api.send("publishers.set_demod_annotations", ["LTE","5G"])
```

Set Demod Threshold (`publishers.set_demod_threshold`)

Sets a threshold for the correlation of demods to active signals. Only detections above the set confidence threshold will be passed onto publishers. Defaults value is 0.8. Must be > 0.0 and < 1.0.

Usage:

```
api.send("publishers.set_demod_threshold", 0.8)
```

Set Demod Tolerance Bins (`publishers.set_demod_tolerance_bins`)

Sets the tolerance used to match up demod information with active signals based on bin width.

Usage:

```
api.send("publishers.set_demod_tolerance_bins", 1.5)
```

Recording

Get Trigger Config (`recording.get_trigger_config`)

Gives the JSON configuration for automatic recording.

Usage:

```
api.send("recording.get_trigger_config")
```

See Automatic Record Mode for more details on each parameter.

Set Capture Length Ms (`recording.set_capture_length_ms`)

Sets the capture length for automatic recordings, in milliseconds. The capture length is the total length of the recorded snapshot. It will default to 1000 ms if not set.

Usage:

```
api.send("recording.set_capture_length_ms", "2000")
```

Set Capture Offset Ms (`recording.set_capture_offset_ms`)

Sets the capture offset for automatic recordings, in milliseconds. The capture offset is how far in advance of the detection to begin recording. It will default to 0 ms if not set.

Usage:

```
api.send("recording.set_capture_offset_ms", "500")
```

Set Descriptions (`recording.set_descriptions`)

Sets the descriptions for automatic recordings. When automatic recording is enabled, detections matching the Signal Descriptions field set here and meeting criteria specified in other fields will be automatically recorded and saved in `capture destination`.

Usage:

```
api.send("recording.set_descriptions", '["LTE","5G"]')
```

Set Max Bandwidth (`recording.set_max_bandwidth`)

Sets the maximum bandwidth for automatic recording, in Hz. All detections with bandwidth (upper_freq-lower_freq) values above the maximum bandwidth will not be automatically recorded.

Usage:

```
api.send("recording.set_max_bandwidth", "10e6")
```

Set Max RSSI (`recording.set_max_rssi`)

Sets the maximum RSSI for automatic recording. All detections with RSSI values above the maximum RSSI will not be automatically recorded.

Usage:

```
api.send("recording.set_max_rssi", "10")
```

Set Min Bandwidth (`recording.set_min_bandwidth`)

Sets the minimum bandwidth for automatic recording, in Hz. All detections with bandwidth (upper_freq-lower_freq) values below the minimum bandwidth will not be automatically recorded.

Usage:

```
api.send("recording.set_min_bandwidth", "5e6")
```

Set Min Confidence (`recording.set_min_confidence`)

Sets the minimum confidence for automatic recording. All detections with confidence values lower than the minimum confidence will not be automatically recorded.

Usage:

```
api.send("recording.set_min_confidence", "0.5")
```

Set Min RSSI (`recording.set_min_rssi`)

Sets the minimum RSSI for automatic recording. All detections with RSSI values above the maximum RSSI will not be automatically recorded.

Usage:

```
api.send("recording.set_min_rssi", "-10")
```

Set Trigger Config (`recording.set_trigger_config`)

Sets the JSON configuration for automatic recording. Trigger record configurations set from API will update the Automatic Recording fields in the OmniSIG Engine visualizer.

Usage:

```
api.send("recording.set_trigger_config", '{"descriptions":
["5G"],"enabled":true,"max_bandwidth":20.0e6,"max_rssi":100.0,"min_bandwidth":5.0
```

When using the utility as an imported module, you may also do (reccomended method):

```
json_dict ={
        "descriptions":["LTE,5G"],
        "enabled":True,
        "max_bandwidth":10.0e6,
        "max_rssi":100.0,
        "min_bandwidth":5.0e6,
        "min_confidence":0.0,
        "min_rssi":-100.0
}

api.send("recording.set_trigger_config", json.dumps(json_dict))
```

When using the utility standalone, no quotes around the JSON are needed:

```
deepsig> recording.set_trigger_config {"descriptions":
["LTE,5G"],"enabled":true,"max_bandwidth":10.0e6,"max_rssi":100.0,"min_bandwidth"
```

Note the lowercase t `true` .

See Automatic Record Mode for more details on each parameter.

Toggle Trigger On Off (`recording.toggle_trigger_on_off`)

Enables automatic recording if it was previously disabled, or disables it if it was previously enabled.

Usage:

```
api.send("recording.toggle_trigger_on_off")
```

Sources

Get Source Type (`source[0].get_type`)

Returns the type of the source, e.g. "uhd" or "sidekiq".

Usage:

```
api.send("source[0].get_type")
```

Get Source Subtype (`source[0].get_subtype`)

Returns the subtype of the source, e.g. "b200" for a source of type "uhd". Defaults to "unknown".

Usage:

```
api.send("source[0].get_subtype")
```

Calibrate (`source[0].calibrate`)

Triggers an internal cross correlation based calibration and immediately updates the loaded weights. This function takes a single parameter which is the length of the correlation in samples.

Get Aperture Bearing (`source[0].get_aperture_bearing`)

Returns the Aperture Bearing object which OmniSIG reports in the Spatial SigMF Field `aperture_bearing`.

Get Aperture Rotation (`source[0].get_aperture_rotation`)

Returns the Aperture Rotation object which OmniSIG reports in the Spatial SigMF Field `aperture_rotation`.

Get Calibration Weights (`source[0].get_calibration_weights`)

Returns the currently loaded calibration weights.

Get Capture Destination (`source[0].get_capture_destination`)

Gets the absolute path on host to the default directory used for raw IQ recordings out of OmniSIG.

Usage:

```
api.send("source[0].get_capture_destination")
```

Get Channelizer Config (`source[0].get_channelizer_config`)

Returns the current channelizer JSON config.

See Channelizer Settings Tab for more details.

Usage:

```
api.send("source[0].get_channelizer_config")
```

Get Element Geometry (`source[0].get_element_geometry`)

Get the antenna element geometry.

Usage:

```
api.send("source[0].get_element_geometry")
```

Get Emitter Bearing (`source[0].get_emitter_bearing`)

Get the emitter bearing.

Usage:

```
api.send("source[0].get_emitter_bearing")
```

Get Flush Samples (`source[0].get_flush_samples`)

Return the sample flush period in ms.

Usage:

```
api.send("source[0].get_flush_samples")
```

Get Multichannel Config (`source[0].get_multichannel_config`)

Returns the multichannel JSON config.

Usage:

```
api.send("source[0].get_multichannel_config")
```

Get Number of RX Channels (`source[0].get_num_rx_chans`)

Returns the number of available RX channels for the current source.

Usage:

```
api.send("source[0].get_num_rx_chans")
```

Get RX Freq (`source[0].get_rx_freq`)

Gets the RX frequency (if applicable, in Hz).

Usage:

```
api.send("source[0].get_rx_freq")
```

Get RX Gain (`source[0].get_rx_gain`)

Gets the RX gain (if applicable).

Usage:

```
api.send("source[0].get_rx_gain")
```

Get RX Rate (`source[0].get_rx_rate`)

Gets the RX rate (if applicable, in MSps).

Usage:

```
api.send("source[0].get_rx_rate")
```

Get Scanner Delay (`source[0].get_scanner_delay`)

Gets the time to pause at each interval (in ms).

Usage:

```
api.send("source[0].get_scanner_delay")
```

Get Scanner Dwells (`source[0].get_scanner_dwells`)

Get the current list of scanner hopping frequencies.

Usage:

```
api.send("source[0].get_scanner_dwells")
```

Get Scanner Start (`source[0].get_scanner_start`)

Gets the lower frequency bound for the radio scanner (in Hz) operating in a linear sweep.

Usage:

```
api.send("source[0].get_scanner_start")
```

Get Scanner State (`source[0].get_scanner_state`)

Returns `True` if OmniSIG is currently operating in scanning mode -- otherwise returns `False`

Usage:

```
api.send("source[0].get_scanner_state")
```

Get Scanner Stop (`source[0].get_scanner_stop`)

Gets the upper frequency bound for the radio scanner (in Hz) operating in a linear sweep.

Usage:

```
api.send("source[0].get_scanner_stop")
```

Load Multichannel Config (`source[0].load_multichannel_config`)

Loads the multichannel config once it has been set explicitly (either by raw JSON or file path)

Usage:

```
api.send("source[0].load_multichannel_config")
```

Measure Channel Offsets (`source[0].measure_channel_offsets`)

Triggers an internal cross correlation based calibration but does not update the loaded weights. This function takes a single parameter which is the length of the correlation in samples.

Measure Phase Offsets (`source[0].measure_phase_offsets`)

Accept a user-defined number of samples to compute the correla$on across as an input.

Usage:

```
api.send("source[0].measure_phase_offsets", 16000)
```

Record Samples Snap (`source[0].record_samples_snap`)

Record I/Q data for specified number of samples.

Usage:

```
api.send("source[0].record_samples_snap","100e6")
```

See Capture Destination for more details.

Record Snap (`source[0].record_snap`)

Provides a more customizable approach to raw I/Q recording. A JSON config in the form of:

```
{
        "decimation_factor": 1,
```

```
        "description": "OmniSIG Recording",
        "antenna_type": "UNK",
        "filename_descriptor": "",
        "method": "manual",
        "indicator": "manual",
        "rule": "manual",
        "endpoint": "<default>",
        "datatype": "ci16_le"
        "samples": <num_samples>,
        "time": <time_length_seconds>
}
```

can be used to configure individual parameters of the I/Q recording. In the above example, the default values if not provided are shown.

NOTE: a value for either `"samples"` or `"time"` is required

Usage:

```
api.send("source[0].record_snap", json.dumps(json_dict))
```

Record Timed Snap (`source[0].record_timed_snap`)

Record I/Q data for specified number of seconds.

Usage:

```
api.send("source[0].record_timed_snap","1.1")
```

See Capture Destination for more details.

Reset Calibration Weights (`source[0].reset_calibration_weights`)

Looks up the calibration weights in the Calibration Table at the current frequency and loads them.

Set Annotations (`source[0].set_annotations`)

Set Antenna Model (`source[0].set_antenna_model`)

Set Antenna Type (`source[0].set_antenna_type`)

Set Aperture Azimuth (`source[0].set_aperture_azimuth`)

Sets the spatial::aperture_azimuth SigMF field. Takes a single argument which is the azimuth in degrees East of true North.

Set Aperture Bearing (`source[0].set_aperture_bearing`)

Sets the spatial::aperture_bearing SigMF object. Takes a single argument which is the raw JSON representing this object.

Set Aperture Elevation (`source[0].set_aperture_elevation`)

Sets the elevation field inside the Aperture Bearing SigMF object. Takes a single argument which is the elevation in degrees above horizon.

Set Aperture Rotation (`source[0].set_aperture_rotation`)

Sets the spatial::aperture_rotation SigMF field. Takes a single argument which is the right-handed rotation of the aperture along the boresight axis.

Set Calibration Weights (`source[0].set_calibration_weights`)

Sets the currently loaded calibration weights to the specified values. Takes a single argument which is a JSON array of complex values each represented as a two element array.

Set Capture Destination (`source[0].set_capture_destination`)

Sets the absolute path on host to the default directory used for raw IQ recordings out of OmniSIG.

Usage:

```
api.send("source[0].set_capture_destination", "/path/to/captures/")
```

Set Element Geometry (`source[0].set_element_geometry`)

Set Emitter Azimuth (`source[0].set_emitter_azimuth`)

Sets the azimuth field within the spatial::emitter_bearing SigMF object. Takes a single argument which is degrees east of true north.

Set Emitter Bearing (`source[0].set_emitter_bearing`)

Sets the spatial::emitter_bearing SigMF object. Takes a single argument which is the raw JSON representing this object.

Set Emitter Elevation (`source[0].set_emitter_elevation`)

Sets the elevation field within the spatial::emitter_bearing SigMF object. Takes a single argument which is elevation in degrees above horizon.

Set Flush Samples (`source[0].set_flush_samples`)

Return the sample flush period in ms.

Usage:

```
api.send("source[0].set_flush_samples", 10)
```

Set Lat Lon Alt (`source[0].set_lat_lon_alt`)

Sets/overwrites the geotag metadata that is included in OmniSIG annotation output (useful if coordinate information is desirable, but otherwise unavailable from a genuine GPS source). Indicated as a comma separated value in the form of `<latitude>,<longitude>,<optional_altitude_in_meters>`

Usage:

```
api.send("source[0].set_lat_lon_alt", "38.91,77.03,10.0")
```

Set Multichannel Config (`source[0].set_multichannel_config`)

Set the multichannel JSON config explicitly.

Usage:

```
json_dict = {
    "calibration_table": [
        {
            "frequency": 860000000.0,
            "phases":[0,0.2318],
            "amplitudes": [1, 0.984]
        },
        {
            "frequency": 900000000.0,
            "phases":[0, 1.0],
            "amplitudes":[1, 0.99]
        }
    ]
}
api.send("source[0].set_multichannel_config", json.dumps(json_dict))
```

Set Multichannel Config Path (`source[0].set_multichannel_config_path`)

Sets the multichannel config via a path to a JSON config file.

Usage:

```
api.send("source[0].set_multichannel_config_path",
"/path/to/multichannel_config.json")
```

Set RX Freq (`source[0].set_rx_freq`)

Sets the RX frequency (if applicable, in Hz).

Usage:

```
api.send("source[0].set_rx_freq","8.8e8")
```

Set RX Gain (`source[0].set_rx_gain`)

Sets the RX gain (if applicable).

Usage:

```
api.send("source[0].set_rx_gain","55")
```

Set RX Rate (`source[0].set_rx_rate`)

Sets the RX rate (if applicable, in MSps).

Usage:

```
api.send("source[0].set_rx_rate","40e6")
```

Set Scanner Delay (`source[0].set_scanner_delay`)

Sets the time to pause at each interval (in ms).

Usage:

```
api.send("source[0].set_scanner_delay", "1000")
```

Set Scanner Dwells (`source[0].set_scanner_dwells`)

Specify a list of frequencies to hop between. The frequencies should be comma-separated and given in MHz. As an example, the following would hop between 800 MHz, 900 MHz, and 1000 MHz.

Usage:

```
api.send("source[0].set_scanner_dwells", "800,900,1000")
```

Set Scanner Start (`source[0].set_scanner_start`)

Sets the lower frequency bound for the radio scanner (in Hz) which will operate in a linear sweep. Must be less than the scanner stop frequency.

Usage:

```
api.send("source[0].set_scanner_start", "700e6")
```

Set Scanner Stop (`source[0].set_scanner_stop`)

Sets the upper frequency bound for the radio scanner (in Hz) which will operate in a linear sweep. Must be greater than the scanner start frequency.

Usage:

```
api.send("source[0].set_scanner_stop", "840e6")
```

Start (`source[0].start`)

Starts a source by unique source ID.

Usage:

```
api.send("source[0].start")
```

Start Scan (`source[0].start_scan`)

Starts/resumes radio scanner.

Usage:

```
api.send("source[0].start_scan")
```

Stop (`source[0].stop`)

Stops/pauses a source by unique source ID.

Usage:

```
api.send("source[0].stop")
```

Stop Scan (`source[0].stop_scan`)

Stops/pauses radio scanner.

Usage:

```
api.send("source[0].stop_scan")
```

Store Active Calibration Weights (`source[0].store_active_calibration_weights`)

Stores the currently loaded calibration weights at the current RF frequency into the calibration weight table for persistent recall.

Store Calibration Entry (`source[0].store_calibration_entry`)

Stores a calibration table entry from JSON. Takes a single argument which is the raw JSON. Format is `{"frequency":50000000.0,"phases":[0, 0.2318],"amplitudes":[1, 0.984]}`

Wideband Inference

Get Confidence Threshold (`wideband_inference.get_confidence_threshold`)

Gets the confidence threshold for the displayed signals. Only detections above this confidence threshold published in OmniSIG output

Usage:

```
api.send("wideband_inference.get_confidence_threshold")
```

Get Demod Signals (`wideband_inference.get_demod_signals`)

Returns a string of signal labels passed to OmniSIG on startup by the `--demod str` parameter.

Usage:

```
api.send("wideband_inference.get_demod_signals")
```

Get Demod Args (`wideband_inference.get_demod_args`)

Returns a string of arguments passed to OmniSIG on startup by the `--demod-args str` parameter.

Usage:

```
api.send("wideband_inference.get_demod_args")
```

Get Min Period (`wideband_inference.get_min_period`)

Gets the minimum allowable time period (in seconds) for OmniSIG Engine to make 1 inference.

Usage:

```
api.send("wideband_inference.get_min_period")
```

Get RSSI Threshold (`wideband_inference.get_rssi_threshold`)

Gets the RSSI value below which annotations are filtered from OmniSIG output.

Usage:

```
api.send("wideband_inference.get_rssi_threshold")
```

Get Signal List (`wideband_inference.get_signal_list`)

Returns a list of all the signal types the current running model has been training to detect and classify.

Usage:

```
api.send("wideband_inference.get_signal_list")
```

Get Static Annotations (`wideband_inference.get_static_annotations`)

Get Throttle Config (`wideband_inference.get_throttle_config`)

Get Unknown Confidence Threshold (`wideband_inference.get_unknown_confidence_threshold`)

Gets the unknown confidence threshold. Any detections above this confidence and below the confidence threshold above will be marked as `Unknown`.

Usage:

```
api.send("wideband_inference.get_unknown_confidence_threshold")
```

Reset RSSI Threshold (`wideband_inference.reset_rssi_threshold`)

Reset/disable the RSSI threshold filtering.

Usage:

```
api.send("wideband_inference.reset_rssi_threshold")
```

Reset Unknown Confidence Threshold (`wideband_inference.reset_unknown_confidence_threshold`)

Reset/disable the unknown confidence threshold filtering

Usage:

```
api.send("wideband_inference.reset_unknown_confidence_threshold")
```

Set Confidence Threshold (`wideband_inference.set_confidence_threshold`)

Sets a confidence threshold for the displayed signals. Only detections above the set confidence threshold will be passed onto publishers. Defaults value is 0.7. Must be > 0.0 and < 1.0.

Usage:

```
api.send("wideband_inference.set_confidence_threshold", "0.7")
```

Set Min Period (`wideband_inference.set_min_period`)

Sets the minimum allowable time period (in seconds) for OmniSIG Engine to make 1 inference.

Usage:

```
api.send("wideband_inference.set_min_period", "0.5")
```

Set RSSI Threshold (`wideband_inference.set_rssi_threshold`)

Sets the RSSI value below which annotations are filtered from OmniSIG output.

Usage:

```
api.send("wideband_inference.set_rssi_threshold", "0.0")
```

Set Static Annotations (`wideband_inference.set_static_annotations`)

Set Static Annotations Only (`wideband_inference.set_static_annotations_only`)

Set Throttle Config (`wideband_inference.set_throttle_config`)

Set Unknown Confidence Threshold (`wideband_inference.set_unknown_confidence_threshold`)

To be used in conjunction with `wideband_inference.set_confidence_threshold`. Any detections above this confidence and below the confidence threshold above will be marked as `Unknown`.

Usage:

```
api.send("wideband_inference.set_unknown_confidence_threshold", "0.3")
```